

CrickIQ: An AI-Based Cricket Analytics and Decision Support System for T20 International Cricket

A PROJECT REPORT

Submitted by

KNAK ANAND THADATHIL	230701139
KASHIF NAZIR	230701144
NAREN SURIYA V S	230701203
NITHESH RAJ M	230701214

*in partial fulfillment for the award of the degree
of*

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY : CHENNAI 600 025

SEPTEMBER 2026

ANNA UNIVERSITY : CHENNAI 600 025

BONAFIDE CERTIFICATE

Certified that this project report “BEYOND CONVERGENCE: A LIGHTWEIGHT HUMAN-IN-THE-LOOP SEMANTIC CONFLICT TRIAGE FRAMEWORK FOR OFFLINE-FIRST CRDT-BASED COLLABORATIVE TEXT EDITING” is the bonafide work of “KNAK ANAND THADATHIL (230701139), KASHIF NAZIR (230701144), NAREN SURIYA V S (230701203) and NITHESH RAJ M (230701214)” who carried out the project work under my supervision.

“This project addresses the following Sustainable Development Goal(s): Goal - No : 9. Description: aims to build resilient infrastructure, promote inclusive and sustainable industrialization, and foster innovation”

SIGNATURE

Dr. J. Manoranjini

HEAD OF THE DEPARTMENT

Department of Computer Science and
Engineering
Rajalakshmi Engineering College
Chennai - 602 105

Submitted for the Project

Mr. Karthikeyan Ramanathan

SUPERVISOR

Assistant Professor

Department of Computer Science and

Engineering

Rajalakshmi Engineering College

Chennai - 602 105

Viva-Voce Examination held on

INTERNAL EXAMINER

SIGNATURE

EXTERNAL EXAMINER

ABSTRACT

Modern T20 cricket asks a lot of its coaches and analysts. They must pick the right eleven, read the pitch, study the opposition and do all of it quickly, often with the relevant information spread across different websites, spreadsheets and notebooks. Collecting and combining that information by hand is slow, and it leaves room for inconsistency. CrickIQ, an AI-Based Cricket Analytics and Decision Support System, is proposed to bring this work into one web platform.

The platform combines player and team statistics, venue analysis, pitch intelligence, opponent analysis, playing XI recommendation, match

prediction, match management, live scoring, tournament management, player development, report generation and a natural-language AI assistant. The playing XI module scores each player on performance, role, match conditions, venue and opposition instead of a single statistic. The pitch module converts grass, moisture, hardness, cracks, bounce, weather and dew into tactical advice. The opponent module highlights the players an opposing side relies on.

Explainable AI based on SHAP is used so that every recommendation comes with the reasons behind it, which lets a coach judge whether to trust it. The system is built with React, TypeScript, TanStack Start, Tailwind CSS, Recharts and Supabase with PostgreSQL, and it provides role-based access for coaches, analysts, management and players. Throughout, the design keeps the final decision with a human: CrickIQ suggests and explains, and the coach decides.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S.MEGANATHAN, B.E, F.I.E.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.**, and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved Principal for his kind support and facilities provided to complete our work in time.

We express our sincere thanks to **Dr.J.MANORANJINI**, Professor and Head of the Department of Computer Science and Engineering for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **KARTHIKEYEN RAMANATHAN**, Assistant Professor, Department of Computer Science and Engineering for his valuable guidance throughout the course of the project. We thank our Project Coordinator, **Dr. S. MADHUSUDHANAN**, Associate Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

KANAK ANAND THADATHIL

KASHIF NAZIR

NAREN SURIYA V S

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO. NO.
	ABSTRACT	iii
	ACKNOWLEDGEMENT	v
	LIST OF TABLES	viii
	LIST OF FIGURES	ix
	LIST OF ABBREVIATIONS	x
1.	INTRODUCTION	1
	1.1 GENERAL	1
	1.2 PROBLEM STATEMENT	2
	1.3 OBJECTIVES	3
	1.4 EXISTING SYSTEM	4
	1.5 PROPOSED SYSTEM	4
2.	LITERATURE SURVEY	6
	2.1 LITERATURE REVIEW	6
	2.1.1 Operational Transformation	6
	2.1.2 Commutative and Conflict-free Replication	6
	2.1.3 Local-first Software	7
	2.1.4 Rich-text CRDTs	7
	2.1.5 Semantic Conflict Models	8
	2.1.6 Revision Intent Analysis	8
	2.1.7 Natural Language Inference	8
	2.1.8 Human-AI Revision	9
	2.2 COMPARISON AND DISCUSSION	10
	2.3 CONCLUSION	11

3.	SYSTEM DESIGN	12
	3.1 SYSTEM ARCHITECTURE	12
	3.2 SYSTEM REQUIREMENTS	13
	3.2.1 Software Requirements	13
	3.2.2 Hardware Requirements	14
	3.3 DATA AND PERSISTENCE DESIGN	14
	3.4 SYSTEM WORKFLOW	15
	3.4.1 Offline Editing and Reconnection	15
	3.4.2 Inspection Collection and Review	15
	3.4.3 Finalization and PDF Export	16
	3.5 SEMANTIC ANALYSIS DESIGN	18
	3.6 API ARCHITECTURE	19
4.	PROJECT DESCRIPTION	20
	4.1 METHODOLOGY	20
	4.2 SYSTEM MODULES	21
	4.3 DETAILED INSPECTION FEATURE	22
	4.3.1 Inspection Workspace and Location Model	22
	4.3.2 Observation Capture and Connectivity Metadata	23
	4.3.3 Complementary Observation Handling	23
	4.3.4 Contradiction Handling and Human Decision	24
	4.3.5 Collaborator Finalization and Export Lock	25
	4.3.6 Exported Inspection Report	26
5.	IMPLEMENTATION AND RESULT DISCUSSION	28
	5.1 IMPLEMENTATION	28
	5.2 EXPERIMENTAL SETUP	30
	5.3 COLLABORATION TEST SCENARIOS	31
	5.4 SYNCHRONIZATION RESULTS	31
	5.5 INSPECTION FEATURE FUNCTIONAL VALIDATION	32
	5.6 SEMANTIC RESEARCH EVALUATION FROM THE APPROVED PAPER	33
	5.7 DISCUSSION	34
	5.8 LIMITATIONS	35

6.	CONCLUSION AND FUTURE WORK	36
6.1	CONCLUSION	36
6.2	FUTURE WORK	37
	APPENDIX 1 - INSPECTION DATA MODEL AND STATUS RULES	38
	APPENDIX 2 - DETAILED TEST CHECKLIST	39
	REFERENCES	40

LIST OF TABLES

1.1	Comparison of the traditional approach and CrickIQ	
3 2.1	Summary of sources and their relevance to CrickIQ	
		6
3.1	Software requirements of the proposed system	9
3.2	Hardware requirements of the proposed system	10
4.1	Pitch parameters and their tactical interpretation	16
4.2	Factors in the player suitability score	16
5.1	Functional validation of the implemented platform	3 6 9 10 16 16 21

LIST OF FIGURES

Figure 1.

1 Existing approach to match preparation	13
Figure 1.2 Overview of the proposed system	16
Figure 3.1 Architecture diagram of the entire system	17
Figure 3.2 Activity diagram for match preparation	18
Figure 3.3 Activity diagram for live match scoring	24
Figure 3.4 Data flow diagram of the system	24
Figure 4.1 Review and decision by the coach	25
Figure 5.1 CrickIQ login interface	25
Figure 5.2 CrickIQ dashboard interface	26
Figure 5.3 Playing XI recommendation and pitch intelligence interface	27
Figure 5.4 Live match scoring interface	28

LIST OF ABBREVIATIONS

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
ML	Machine Learning
XAL	Explainable Artificial Intelligence
SHAP	SHapley Additive exPlanations
T20I	Twenty20 International
ICC	International Cricket Council
XI	Playing eleven of a team
RBAC	Role-Based Access Control
API	Application Programming Interface
UI	User Interface
SQL	Unified Modeling Language
DFD	Data Flow Diagram
UML	Unified Modeling Language
SDG	Sustainable Development Goal

CHAPTER 1 INTRODUCTION

1.1 GENERAL

Cricket is one of the most data-heavy sports in the world. Every delivery leaves a record: who bowled, who faced, how many runs came, whether a wicket fell, on which ground and on what kind of pitch. Over a season this becomes an enormous body of information about players, teams and venues. The T20 format has made that information even more valuable, because matches are short and a single over can change the result.

For a long time, picking a team and planning tactics depended mostly on the experience and instinct of coaches and analysts. That experience still matters a great deal. What has changed is the amount of data sitting alongside it. Player statistics, venue records, match results and notes on the opposition usually live in different places, and before a decision can be made someone has to gather them, compare them and hold all of it in their head at once.

Artificial intelligence and data analytics offer a practical way to lighten that load. A decision-support system can gather the scattered information, apply it to the situation at hand and present a clear suggestion together with the reasons for it. It does not replace the coach. It gives the coach a well-organized starting point so that more time goes into thinking and less into searching.

1.2 OBJECTIVE

The main aim of CrickIQ is to provide one web platform where T20 cricket data is turned into practical, explainable support for coaches and analysts. It is meant to reduce the manual effort of collecting and comparing information, and to keep the final judgment with the human decision-maker.

The specific objectives are listed below.

1. To provide detailed batting, bowling and fielding statistics for players, and team-level information on squads and roles.
2. To recommend a suitable playing XI by combining performance, role, pitch conditions, venue characteristics and opposition.
3. To convert pitch parameters such as grass, moisture, hardness, cracks, bounce, weather and dew into tactical guidance.
4. To provide historical venue information that supports match preparation.
5. To identify key opposition players and summarize an opponent's strengths and weaknesses.
6. To estimate the relative chances of two teams as an analytical estimate and not a guaranteed result.
7. To use Explainable AI (SHAP) so that recommendations come with the reasons behind them.
8. To support match management, live scoring, tournament management and player development in the same platform.
9. To let users ask questions in natural language through an AI assistant, and to export reports.
10. To provide secure, role-based access for coaches, analysts, management and players.

Blindly accepting both statements into a final report is undesirable, but automatically allowing an AI model to delete or rewrite disputed human observations is also unsafe. Therefore the project requires a mechanism that preserves offline work, detects semantic relationships after synchronization, exposes uncertainty, records human decisions and prevents formal report export until review and finalization are complete.

1.3 EXISTING SYSTEM

In most teams today, match preparation is stitched together from several unconnected tools. Player statistics are read from public statistics websites or kept in spreadsheets. Venue records are gathered separately. Pitch reports come from curators, commentary or the analyst's own observation. Notes on the opposition sit in documents and slide decks, and match scoring is done in yet another application. Figure 1.1 shows this typical workflow.

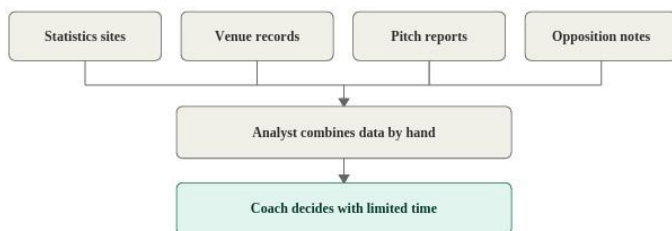


Figure 1.1. Existing approach to match preparation

An analyst in this setup opens each source, copies the relevant figures into a working document, compares them and then summarizes the result for the coach. It works, but it has clear weaknesses. Gathering the data takes time that could be spent on analysis, and important details may be missed simply because they live somewhere nobody thought to look.

A deeper problem is that existing sources mostly report numbers without connecting them to decisions. A batting average does not say whether a player suits a slow, turning pitch. A venue's record does not say what to do about today's conditions. Choosing an XI by hand also means weighing many factors mentally, so two coaches, or the same coach on two different days, may weigh them differently, and there is rarely a reviewable record of why one player was preferred over another. Table 1.1 compares the usual approach with the one proposed in this project.

Table 1.1. Comparison of the traditional approach and CrickIQ

Aspect	Traditional approach	CrickIQ
Primary purpose	Record and report statistics	Support tactical decisions
Data sources	Several separate tools and sheets	One shared database
Context handling	Applied mentally by the coach	Pitch, venue and opponent built in
Team selection	Manual and experience-based	Scored suggestion, coach decides
Explanation of advice	Rarely recorded	SHAP-based reasons shown
Pitch assessment	Informal judgment	Structured inputs to tactical advice
Opponent preparation	Private notes and documents	Key threats highlighted in-app
Match scoring	Separate scoring tool	Integrated live scoring
User interaction	Menus and manual searching	Natural-language AI assistant

1.4 EXISTING SYSTEM

CrickIQ is designed as a single web platform that takes a coach from raw data to an explained recommendation. Its modules share one database, so player information entered once is used by the analytics, selection, opponent and development modules alike. Figure 1.2 gives an overview.

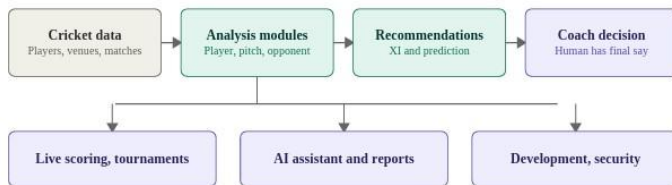


Figure 1.2. Overview of the proposed system

The system begins with cricket data for players, teams, matches and venues, stored in a PostgreSQL database through Supabase. The analysis modules then work on that data. The player and team modules describe form, roles and balance. The pitch and venue modules supply context, and the opponent module

highlights the threats and weaknesses of the other side. The decision-support stage combines these into a playing XI recommendation and a match prediction.

Two design choices shape the whole system. The first is explainability: the AI model approach adopted for the project is Explainable AI using SHAP, so that the coach can see which factors pushed a player up or down the ranking instead of receiving an unexplained score. The second is human authority: every recommendation is a suggestion that the coach can accept, modify or reject. Around this core, CrickIQ also offers match management, live scoring, tournament management, player development, reports and a natural-language AI assistant, with role-based access for four types of user.

CHAPTER 2

LITERATURE SURVEY

This chapter reviews the literature and sources that support the ideas behind CrickIQ: datadriven decision making, statistical and machine-learning methods, explainable AI, the rules and statistics of T20 cricket, and the web technologies used to build the platform.

2.1 LITERATURE REVIEW

Davenport and Harris [3] argued that organizations gain a lasting advantage when they use analytics systematically and build it into everyday decisions, rather than treating it as an occasional specialist exercise. Their central message fits cricket closely. A coach does not need a research report once a season; a coach needs usable answers every week, and sometimes every day. This work motivates the idea that a decision-support platform should sit inside the normal preparation routine. The technical foundations for turning data into predictions are set out in the standard texts on data mining and statistical learning by Witten et al. [4], Hastie et al. [5] and Han et al. [6]. They describe how to prepare data, how to choose and evaluate models, and how to avoid traps such as overfitting. Although they are not specific to sport, their methods apply directly to sports data, for example when estimating the probability of a result or grouping similar performances. They also remind us that a model is only as good as the data behind it, which is why CrickIQ puts so much weight on clean, shared data. Two algorithms stand out for structured data of the kind cricket produces. Breiman's random forests [7] combine many decision trees to give stable predictions that resist overfitting and indicate which inputs matter most. Chen and Guestrin's XGBoost [8] is a scalable implementation of gradient-boosted trees and is widely used for tabular problems. Both are natural candidates for predicting match outcomes or forecasting player performance in later versions of CrickIQ. Because coaches will only act on advice they understand, explainability is important. Lundberg

and Lee [11] introduced SHAP, a unified approach that attributes a model's prediction to its input features using ideas from cooperative game theory. In CrickIQ, this allows a recommendation to be explained in terms of factors such as performance, role, conditions, venue and opposition, so the coach can see why a player was ranked where he was. The International Cricket Council publishes the playing conditions that govern T20 internationals [1] and maintains rankings and statistical records for teams and players [2]. These sources define the format and supply the kind of raw figures that any cricket analytics tool builds on. They describe what happened and what the rules are, but they do not tell a coach what to do next, which is the gap CrickIQ tries to fill. On the engineering side, the documentation for React [9] describes the component-based interface model used to build the front end, and the documentation for Supabase [10] describes the managed PostgreSQL database, authentication and access-control features used for the back end. Modern web frameworks of this kind make it practical for a small team to build a full analytics application. Table 2.1 summarizes how each source contributes to the project.

2.2 COMPARISON AND DISCUSSION

Taken together, the sources provide a case for analytics-driven decisions, a toolbox of statistical and machine-learning methods, a method for explaining model output, and official sources of cricket data and rules. Most of them concentrate on one piece of the puzzle. The data sources report numbers, the textbooks explain methods, and the web documentation explains tools. What they do not provide is a single, practical platform that applies these ideas to the context-heavy decisions of T20 coaching, in a form that nontechnical users can operate. CrickIQ integrates player analysis, pitch and venue intelligence, opponent analysis, XI recommendation, prediction and explanation in one place, and it keeps the coach in charge of the final call.

2.3 CONCLUSION

The literature shows that analytics, machine learning and explainable AI are mature enough to support sport-related decisions, and that the required web technology is readily available. It also shows a gap: there is little material on an integrated, explainable platform for T20 selection and tactics. CrickIQ addresses this gap by combining contextual analysis, XI recommendation, match prediction and SHAP-based explanation with human review.

CHAPTER 3

SYSTEM DESIGN

This chapter describes the design of the proposed system through diagrams. It covers the overall architecture, the software, hardware and data requirements, and the activity and data flow diagrams that explain how the system behaves.

3.1 SYSTEM ARCHITECTURE

CrickIQ follows a layered, full-stack web architecture, shown in Figure 3.1. Users such as coaches, analysts, management staff and players reach the platform through a web interface. The presentation layer is built with React and TypeScript and uses Tailwind CSS for responsive styling and Recharts for charts. It provides the dashboards, forms and tables through which all modules are used.

The data and services layer has three parts. Supabase Authentication handles login and works with database security policies to provide role-based access. PostgreSQL, provided through Supabase, stores teams, players, statistics, matches, venues, squads, innings, ball events and tournaments. The AI assistant lets users put questions to the system in plain language and draws on the same stored data. Four roles use the system. Coaches work with player analysis, XI recommendation, pitch intelligence, opponent analysis and match analysis. Analysts work with statistics, comparisons, team analysis, match information and reports. Management works with teams, players, tournaments, reports and overall analytics, and players see the performance and development information relevant to them.

3.2 SYSTEM REQUIREMENTS

This section lists the software, hardware and data needed to build and run the proposed system.

3.2.1 SOFTWARE REQUIREMENTS

The platform is written in TypeScript and runs on React, with TanStack Start providing the application framework and Vite serving as the build tool. Tailwind CSS handles styling, and Recharts draws the charts. Supabase provides the PostgreSQL database and the authentication service. Zod validates user input, while jsPDF and XLSX generate PDF and spreadsheet reports. The SHAP library supports the explainable AI component. Table 3.1 lists the main components.

Table 3.1. Software requirements of the proposed system

Software component	Suggested version	Purpose
React and TypeScript	Latest stable	Builds the interactive user interface
TanStack Start	Latest stable	Application framework and routing
Tailwind CSS	Latest stable	Responsive styling
Recharts	Latest stable	Interactive charts and dashboards
Supabase (PostgreSQL)	Latest stable	Database, authentication and access policies
Zod	Latest stable	Validation of entered data
jsPDF and XLSX	Latest stable	PDF and spreadsheet reports
SHAP	Latest stable	Explains the factors behind recommendations
Vite	Latest stable	Build tool for development and production
Visual Studio Code and Git	Latest stable	Development and version control

3.2.2 HARDWARE REQUIREMENTS

CrickIQ is a web application, so the hardware needed for development and normal use is modest. A multicore processor and 8 GB of memory are enough for development, although 16 GB is more comfortable when the database and other services run locally. A solid-state drive keeps the database and build tools responsive. A GPU is not required for normal use, and it would only matter if heavier machine-learning models are trained in future. Table 3.2 lists the suggested specification.

Table 3.2. Hardware requirements of the proposed system

Hardware component	Suggested specification	Purpose
Processor	Intel Core i5 / AMD Ryzen 5 or above	Runs the application and database tools
RAM	8 GB minimum, 16 GB recommended	Runs services and development tools together
Storage	256 GB SSD or more	Stores source code, data and dependencies
Display	Full HD screen	Dashboards and charts
Network	Stable broadband or Wi-Fi	Needed for the cloud database and authentication

3.2.3 DATASET REQUIREMENTS

The system needs the following kinds of data. Together they feed the analytics, selection, pitch, opponent and prediction modules. 1. Player data: name, country, team, role, batting and bowling style, matches, runs, batting average, strike rate, wickets, economy rate, bowling average and fielding information. 2. Team data: team name, country, rating and squad composition. 3. Venue data: average first-innings score, pace and spin suitability, pitch type, boundary dimensions, chase percentage and toss trends. 4. Match data: teams, venue, status, date and time, toss, result and score. 5. Pitch observations: grass, moisture, hardness, cracks, bounce, weather and dew, entered by the user before a match.

3.2.4 DEPLOYMENT AND SCALING

The application can be deployed on a cloud platform with Supabase hosting the database and authentication. As the number of users and the volume of match data grow, the managed database can be scaled without changes to the application code. Because the system is a web application, coaches and analysts need nothing more than a browser.

3.3 SYSTEM DESIGN

This section uses activity and data flow diagrams to show how the system works and how its parts communicate.

3.3.1 ACTIVITY DIAGRAM

Figure 3.2 shows the main activity flow for preparing a match. After logging in, the user selects the match, venue and squad and enters the pitch conditions. The system analyses the players and the opposition and generates a playing XI and a match prediction along with an explanation. The coach then decides whether he is satisfied. If he is, the final XI is saved and a report can be exported. If not, the selection or the inputs are adjusted and the recommendation is generated again.

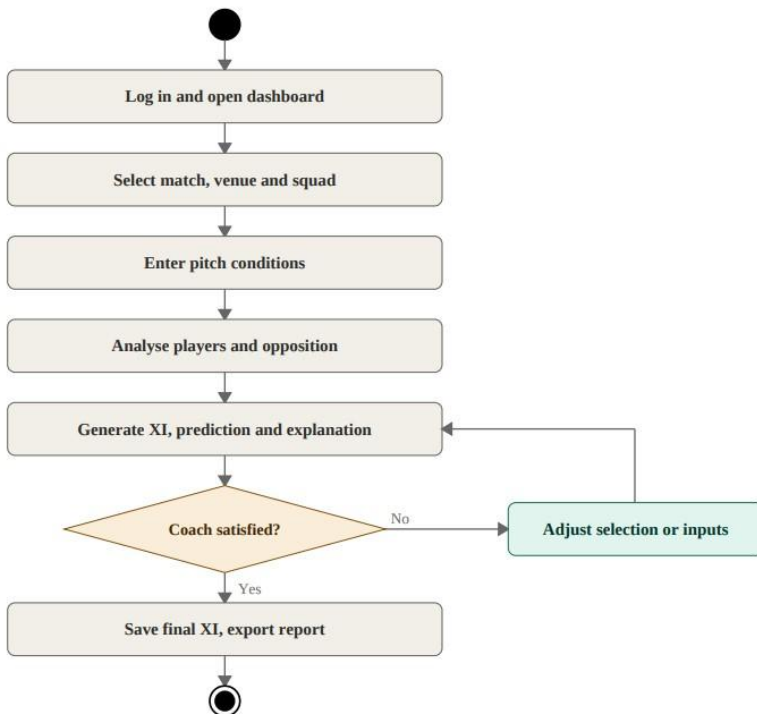


Figure 3.2. Activity diagram for match preparation

Live match scoring follows a simpler loop, shown in Figure 3.3. Each delivery is recorded as an event covering runs, wickets and extras, and the score and overs update immediately. The loop repeats until the innings is complete, after which the final scorecard and result are shown.

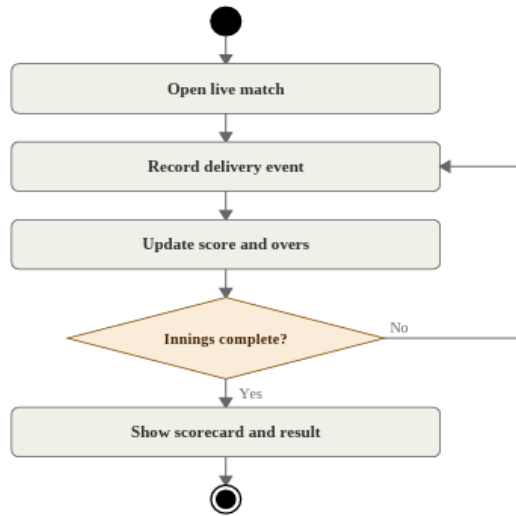


Figure 3.3. Activity diagram for live match scoring

3.3.2 DATAFLOW DIAGRAM

The data flow diagram in Figure 3.4 shows how data moves through the system. Cricket data is stored in the database, cleaned and aggregated, and then passed to the player and team analysis. The results feed the contextual analysis, where pitch, venue and opponent information are applied. The decision-support stage then produces the playing XI suggestion and the match prediction, along with SHAP-based explanations, which reach the coach for the final decision.

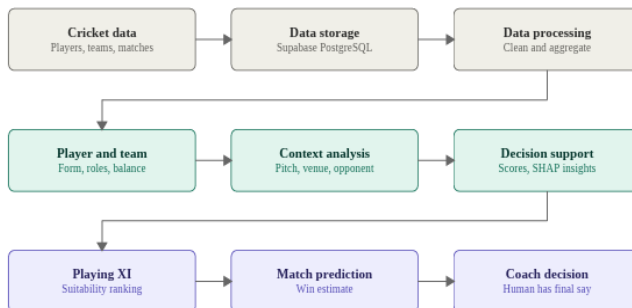


Figure 3.4. Data flow diagram of the system

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

CrickIQ is built around three ideas. The first is integration: everything a coach needs should be in one place and drawn from one shared source. The second is context: no statistic is judged on its own, so every recommendation takes the pitch, the venue and the opposition into account. The third is human authority: the system explains its suggestions, and the coach decides.

In practice, the method is a pipeline. Cricket data is stored and processed. Player and team analysis describes what each side has available. Contextual analysis adds the pitch, venue and opponent. The decision-support stage produces a playing XI and a match prediction, and Explainable AI using SHAP shows which factors drove each result. Finally the coach reviews the output, accepts or changes it, and saves the decision.

4.2 MODULES

The system is divided into the following modules. Each performs one function and uses the shared database.

4.2.1 PLAYER AND TEAM DATA MANAGEMENT

This module keeps the player and team records that every other module depends on. For each player it stores name, country, team, playing role, batting and bowling style, matches, runs, batting average, strike rate, wickets, economy rate, bowling average and fielding information. For each team it stores the name, country, rating and squad. Users can search and compare players, and charts make differences easy to see. The data is validated on entry so that incomplete or incorrect records do not affect later analysis.

4.2.2 PITCH AND VENUE INTELLIGENCE

The pitch intelligence module takes seven inputs: grass, moisture, hardness, cracks, bounce, weather and dew. It interprets them with widely understood cricket reasoning and returns plain tactical advice. Grass tends to help seam bowlers, a hard surface with good bounce favours pace and stroke play, and a slow surface makes spin and variations more effective. Dew can reduce the bowlers' grip in the second innings. Table 4.1 summarizes these relationships.

Table 4.1. Pitch parameters and their tactical interpretation

Pitch condition	Likely effect	Tactical implication
Good grass cover	Assists seam movement	Consider extra seam bowling
Hard surface, good bounce	Favours pace and stroke play	Pace options, attacking batting
Slow, dry surface	Makes spin and variation effective	Consider extra spin
Cracks, wear	Uneven bounce, turn	Spin bowling gains value
Dew	Less grip for bowlers in the second innings	Chasing may be easier

The venue module adds long-term background. It stores the average first-innings score, pace and spin suitability, pitch type, boundary dimensions, chase percentage and toss trends. Combining this history with the current pitch reading lets the coach judge, for instance, whether extra batting depth or an additional spinner is worth considering.

4.2.3 PLAYING XI RECOMMENDATION

Choosing an XI is a multi-factor decision, so CrickIQ scores every available player on several dimensions. The conceptual suitability score of player i is

$$S_i = w_1 P_i + w_2 R_i + w_3 C_i + w_4 V_i + w_5 O_i \quad (4.1)$$

Table 4.2 explains each term. The weights can be adjusted to reflect the coach's priorities. A coach who cares mostly about current form raises the weight on performance, while one preparing for a spin-friendly ground raises the weights on conditions and venue. The system then proposes a combination that also checks batting depth, bowling options and role balance.

Table 4.2. Factors in the player suitability score

Symbol	Factor	Meaning
P_i	Performance	Current statistics of the player
R_i	Role suitability	How well the role fits the team's needs
C_i	Condition suitability	Fit with pitch and weather conditions
V_i	Venue suitability	Fit with the ground's history and dimensions
O_i	Opponent suitability	Usefulness against this opposition
w_1 to w_5	Weights	Importance given to each factor

4.2.4 OPPONENT ANALYSIS AND MATCH PREDICTION

The opponent module lets users examine an opposing side's players, roles, composition, batting and bowling strengths and matchups. It highlights key threats, such as an in-form batter, so that bowling plans can be prepared, and it points out dangerous bowlers so that batting plans can be made in advance.

The prediction module estimates the relative chances of two teams. In simplified form it is expressed as

$$P(W) = f(T, P, V, C) \quad (4.2)$$

where T is team strength, P is player performance, V is venue characteristics and C is match conditions. The output is an analytical estimate and is always presented that way. Cricket is uncertain, and one event can turn a match, so the estimate is meant as an extra statistical perspective for preparation and not as a forecast.

4.2.5 EXPLAINABLE AI AND HUMAN REVIEW

A recommendation is only useful if the coach can judge whether to trust it. For that reason, SHAP is used to attribute each recommendation or prediction to the factors behind it, so the coach can see what pushed a player up or down the ranking. The explanation is shown with the suggestion.

The coach then reviews the result, as shown in Figure 4.1. He can accept the suggestion, modify the selection, or change the inputs and run the analysis again. The decision stays with him, and the final XI can be saved and exported.

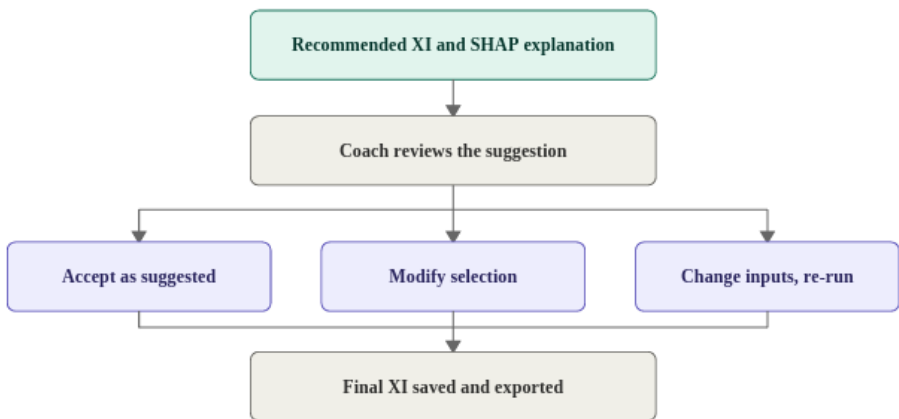


Figure 4.1. Review and decision by the coach

4.2.6 MATCH MANAGEMENT, LIVE SCORING AND TOURNAMENTS

The matches module stores teams, venue, status, date, toss, result and score for scheduled and ongoing matches. The live scoring module lets users record runs, wickets, extras, overs, deliveries and commentary, and maintains a running scorecard. The tournament module supports creating tournaments, adding teams, generating fixtures and tracking standings, points, wins and losses, so organizers need not schedule by hand.

4.2.7 PLAYER DEVELOPMENT, AI ASSISTANT AND REPORTS

The player development module presents performance information, highlights areas for improvement such as batting, bowling, impact and trends, and links them to training-oriented suggestions. The AI assistant lets users ask questions such as “Compare these two players” or “What does the pitch report indicate?” in plain language. The reports module exports player, match, tournament and statistical reports as PDF or spreadsheet files for coaching meetings and reviews.

4.2.8 SECURITY AND ACCESS CONTROL

Supabase Authentication handles user login, and database security policies restrict data to authorized users. Combined with role-based access, this means each person sees only the functions meant for their role, and management features are protected from unauthorized use.

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION AND RESULT

CrickIQ was implemented as a web application. The interface is built with React and TypeScript on the TanStack Start framework and styled with Tailwind CSS. Recharts renders the charts, and Supabase supplies the PostgreSQL database and authentication. Zod validates inputs, and jsPDF and XLSX generate reports. All modules read from the same database, so a change made in one place appears everywhere.

5.1.1 USER INTERFACE AND ACCESS

Users sign in through the login screen and land on a dashboard that matches their role. Coaches, analysts, management staff and players each see the functions relevant to them. The dashboard gives a quick overview of teams, players and matches, and the navigation leads to the analytics, selection, pitch intelligence, opponent analysis, matches, live match, tournament, development, AI assistant and report pages. The login and dashboard screens are shown in Figures 5.1 and 5.2.

Figure 5.1. CrickIQ login interface

Figure 5.2. CrickIQ dashboard interface

5.1.2 SELECTION AND CONTEXTUAL ANALYSIS

In the selection module, the coach chooses the match, venue and squad and enters the pitch observations. The system calculates suitability scores, proposes a playing XI and shows the reasons behind the ranking. The pitch intelligence screen turns the entered conditions into tactical advice, while the opponent

screen highlights the players most likely to influence the match. Figure 5.3 shows the selection and pitch intelligence screens.

Figure 5.3. Playing XI recommendation and pitch intelligence interface

5.1.3 MATCH MANAGEMENT AND LIVE SCORING

The matches page lists scheduled and ongoing matches with their venue, status, toss and result. In the live match screen, each delivery is entered as an event and the scorecard updates at once. Tournament pages show fixtures and standings. Figure 5.4 shows the live scoring interface.

Figure 5.4. Live match scoring interface

5.1.4 FUNCTIONAL VALIDATION

Each module is validated by checking that it performs its intended function. Table 5.1 lists the validation focus for each module. The last column is left for the observed result, which should be filled in from your own test runs.

Table 5.1. Functional validation of the implemented platform

Module	Validation focus	Observed result
Authentication	User login and role-based access	[Fill after testing]
Player analytics	Statistics display and comparison	[Fill after testing]
Team analysis	Squad composition and team information	[Fill after testing]
Playing XI selection	XI generation from roles and conditions	[Fill after testing]
Pitch intelligence	Pitch inputs converted to tactical advice	[Fill after testing]
Venue analysis	Venue characteristics and trends	[Fill after testing]
Opponent analysis	Identification of key opposition players	[Fill after testing]
Match prediction	Estimate of relative winning chances	[Fill after testing]
Explainability	SHAP reasons shown with recommendations	[Fill after testing]
Matches	Schedule, toss, status and result	[Fill after testing]
Live scoring	Runs, wickets, extras and scorecard updates	[Fill after testing]
Tournaments	Fixtures and standings tracking	[Fill after testing]
Player development	Identification of improvement areas	[Fill after testing]
AI assistant	Natural-language query responses	[Fill after testing]
Reports	PDF and spreadsheet report generation	[Fill after testing]

5.2 DISCUSSION

The implemented prototype shows that cricket analytics and decision-support functions can live comfortably in a single web platform. Users see the relevant information through one interface instead of searching several sources, and the time saved on gathering data can be spent on thinking about it.

The playing XI module moves the system from describing players to recommending combinations. This matters because overall statistics do not show whether a player suits a particular match. The pitch and venue modules add context that is influential but rarely standardized, and the opponent module lets coaches consider the tactical requirements before the match begins. The prediction module adds a statistical perspective, although it cannot guarantee any result. Explainable AI makes the recommendations easier to trust and to challenge, and the live scoring and AI assistant modules show that scoring and plain-language access can live in the same application.

There are limitations. The quality of the results depends on the quality and completeness of the data. The current version does not use a licensed live data feed, so match information is entered inside the application. Pitch intelligence depends partly on what users enter, and confidence, injuries, communication and psychology are hard to capture in statistics. The system also does not analyze match video. These are the reasons the final decision stays with the coach, and they point to the future work described in the next chapter.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project presented CrickIQ, an AI-based cricket analytics and decision support system for T20 international cricket. It brings player analytics, team analysis, playing XI recommendation, pitch and venue intelligence, opponent analysis, match prediction, match management, live scoring, tournament management, player development, reports and an AI assistant into one web platform.

Its main contribution is the integration of statistical and contextual information into one decision-support system. Instead of relying on individual player statistics alone, CrickIQ considers roles, conditions, venue and opposition, and it uses SHAP-based explainability to show why a recommendation was made. The system is designed to assist coaches, not replace them: the suggestion, its explanation and the final decision all stay visible and under human control.

The project shows how artificial intelligence, modern web technology and sports analytics can support cricket management, in line with the goal of building innovative, data-driven infrastructure. With better data sources and stronger models, the platform can grow into a broader cricket intelligence system.

6.2 FUTURE WORK

Several improvements could be made in later versions.

- **External data integration.** A licensed cricket API could supply live scores, fixtures, playing XI information, player statistics and ball-by-ball data, reducing manual entry.

- **Advanced machine learning.** Models such as random forests [7] and gradient boosting [8] could be used for match prediction, performance

forecasting and player selection, and the weights in Equation 4.1 could be learned from historical outcomes.

- **Computer vision.** Video analysis could detect shots, study bowling actions and track the ball and fielders.

- **Long-term player development.** Performance could be tracked across many matches to flag lasting improvement or decline.
- **Tactical recommendations.** The system could suggest bowling changes, field placements, batting order and matchups during a match.

- **Mobile application.** A mobile version would let coaches and analysts use the platform from the ground.

- **Wider evaluation.** Larger user studies with coaches and analysts, with measured performance and accuracy tests, would show the real benefit of the platform.

REFERENCES

Under normal connectivity, edits were reflected on collaborating clients without a page refresh. The diagnostics panel reported the document state as consistent and the draft records a median synchronization latency of 71 ms during the test session. Across the evaluated offline/reconnection scenarios, 98 merge attempts were recorded and all completed successfully, corresponding to a 100% merge success rate in the reported test set.

Offline persistence was verified by continuing to edit a disconnected client, refreshing the browser while offline and closing/reopening the document before reconnection. Locally entered content remained available. When connectivity returned, offline and remote changes were merged and the diagnostics status returned to synchronized/consistent. A separate room-isolation test confirmed that updates from one document room did not appear in another.

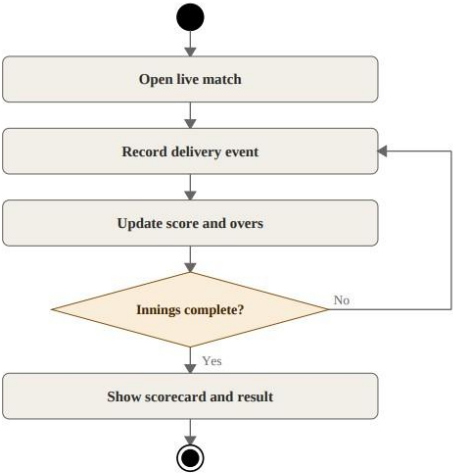


Figure 3.3. Activity diagram for live match scoring

3.3.2 DATAFLOW DIAGRAM

The data flow diagram in Figure 3.4 shows how data moves through the system. Cricket data is stored in the database, cleaned and aggregated, and then passed to the player and team analysis. The results feed the contextual analysis, where pitch, venue and opponent information are applied. The decision-support stage then produces the playing XI suggestion and the match prediction, along with SHAP-based explanations, which reach the coach for the final decision

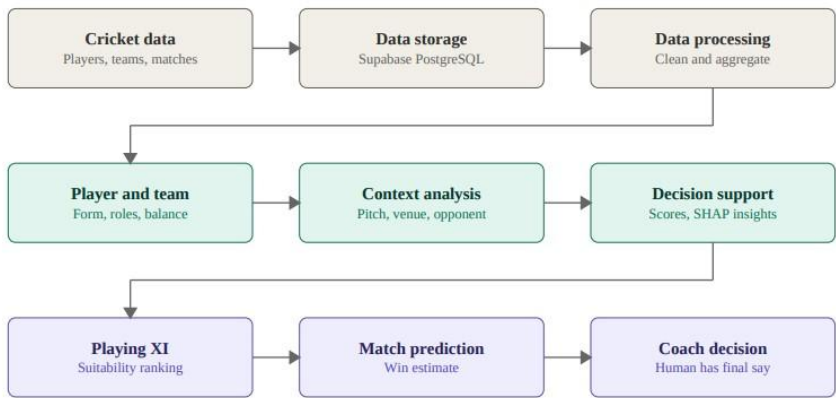


Figure 3.4. Data flow diagram of the system

CHAPTER 4

Methodology and Module Description

4.1 Data Preparation

The engine works on three years of international data for five teams. [State the source, the date range, and the number of matches and players.] Each player has overall statistics for the last three years and for the career, plus phase-wise statistics split by Powerplay, middle and death overs against pace and spin. Each venue carries an average first-innings score and pace and spin bias indices. Before loading, [describe your cleaning: missing values, matching player names across sources, unit checks]. New data enters through CSV import and through the scoring console.

4.2 MODULES

The system is divided into the following modules. Each performs one function and uses the shared database.

4.2.1 PLAYER AND TEAM DATA MANAGEMENT

This module keeps the player and team records that every other module depends on. For each player it stores name, country, team, playing role, batting and bowling style, matches, runs, batting average, strike rate, wickets, economy rate, bowling average and fielding information. For

each team it stores the name, country, rating and squad. Users can search and compare players, and charts make differences easy to see The data is validated on entry so that incomplete or incorrect records do not affect later analysis.

4.2.2 PITCH AND VENUE INTELLIGENCE

The pitch intelligence module takes seven inputs: grass, moisture, hardness, cracks, bounce, weather and dew. It interprets them with widely understood cricket reasoning and returns plain tactical advice. Grass tends to help seam bowlers, a hard surface with good bounce favours pace and stroke play, and a slow surface makes spin and variations more effective. Dew can reduce the bowlers' grip in the second innings. Table 4.1 summarizes these relationships.

Table 4.1. Pitch parameters and their tactical interpretation

Pitch condition	Likely effect	Tactical implication
Good grass cover	Assists seam movement	Consider extra seam bowling
Hard surface, good bounce	Favours pace and stroke play	Pace options, attacking batting
Slow, dry surface	Makes spin and variation effective	Consider extra spin
Cracks, wear	Uneven bounce, turn	Spin bowling gains value
Dew	Less grip for bowlers in the second innings	Chasing may be easier

The venue module adds long-term background. It stores the average firstinnings score, pace and spin suitability, pitch type, boundary dimensions, chase percentage and toss trends. Combining this history with the current pitch reading lets the coach judge, for instance, whether extra batting depth or an additional spinner is worth considering.

4.2.3 PLAYING XI RECOMMENDATION

Choosing an XI is a multi-factor decision, so CrickIQ scores every available player on several dimensions. The conceptual suitability score of player i is

$$S_i = w_1 P_i + w_2 R_i + w_3 C_i + w_4 V_i + w_5 O_i \quad (4.1)$$

Table 4.2 explains each term. The weights can be adjusted to reflect the coach's priorities. A coach who cares mostly about current form raises the weight on performance, while one preparing for a spin-friendly ground raises the weights on conditions and venue. The system then proposes a combination that also checks batting depth, bowling options and role balance.

Table 4.2. Factors in the player suitability score

Symbol	Factor	Meaning
P_i	Performance	Current statistics of the player
R_i	Role suitability	How well the role fits the team's needs
C_i	Condition suitability	Fit with pitch and weather conditions
V_i	Venue suitability	Fit with the ground's history and dimensions
O_i	Opponent suitability	Usefulness against this opposition
w_1 to w_5	Weights	Importance given to each factor

4.2.4 OPPONENT ANALYSIS AND MATCH PREDICTION

The opponent module lets users examine an opposing side's players, roles, composition, batting and bowling strengths and matchups. It highlights key threats, such as an in-form batter, so that bowling plans can be prepared, and it points out dangerous bowlers so that batting plans can be made in advance. The prediction module estimates the relative chances of two teams. In simplified form it is expressed as $P(W) = f(T, P, V, C)$ (4.2) where T is team strength, P is player performance, V is venue characteristics and C is match conditions. The output is an analytical estimate and is always presented that way. Cricket is uncertain, and one event can turn a match, so the estimate is meant as an extra statistical perspective for preparation and not as a forecast.

4.2.5 EXPLAINABLE AI AND HUMAN REVIEW

A recommendation is only useful if the coach can judge whether to trust it. For that reason, SHAP is used to attribute each recommendation or prediction to the factors behind it, so the coach can see what pushed a player up or down the ranking. The explanation is shown with the suggestion. The coach then reviews the result, as shown in Figure 4.1. He can accept the suggestion, modify the

selection, or change the inputs and run the analysis again. The decision stays with him, and the final XI can be saved and exported.

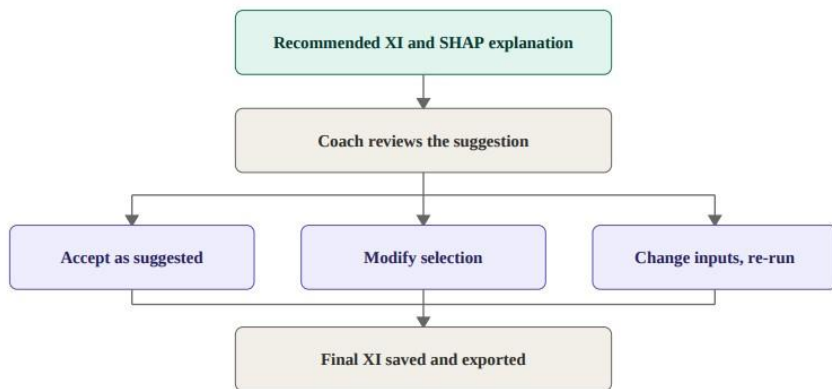


Figure 4.1. Review and decision by the coach

4.2.6 MATCH MANAGEMENT, LIVE SCORING AND TOURNAMENTS

The matches module stores teams, venue, status, date, toss, result and score for scheduled and ongoing matches. The live scoring module lets users record runs, wickets, extras, overs, deliveries and commentary, and maintains a running scorecard. The tournament module supports creating tournaments, adding teams, generating fixtures and tracking standings, points, wins and losses, so organizers need not schedule by hand.

4.2.7 PLAYER DEVELOPMENT, AI ASSISTANT AND REPORTS

The player development module presents performance information, highlights areas for improvement such as batting, bowling, impact and trends, and links them to training-oriented suggestions. The AI assistant lets users ask questions such as “Compare these two players” or “What does the pitch report indicate?” in plain language. The reports module exports player, match, tournament and statistical reports as PDF or spreadsheet files for coaching meetings and reviews.

4.2.8 SECURITY AND ACCESS CONTROL

Supabase Authentication handles user login, and database security policies restrict data to authorized users. Combined with role-based access, this means

each person sees only the functions meant for their role, and management features are protected from unauthorized use.

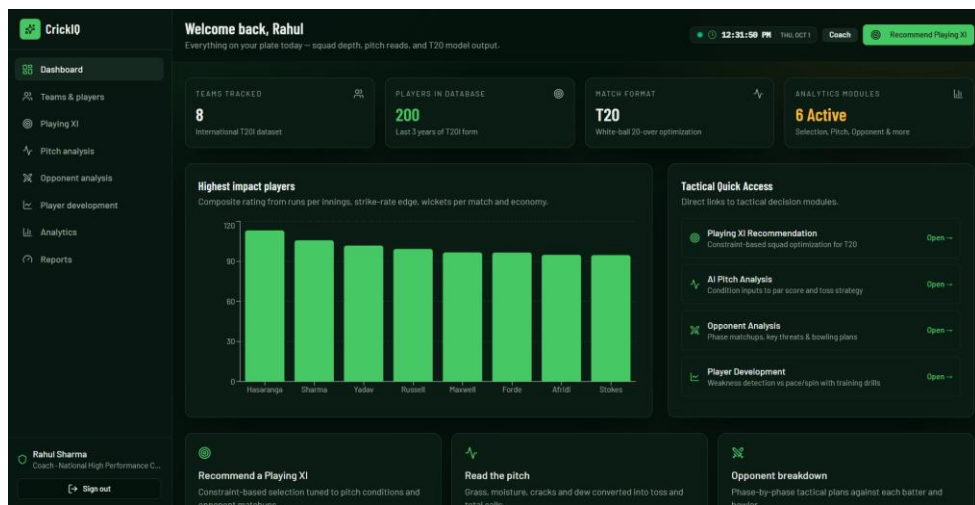
CHAPTER 5 IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION AND RESULT

CrickIQ was implemented as a web application. The interface is built with React and TypeScript on the TanStack Start framework and styled with Tailwind CSS. Recharts renders the charts, and Supabase supplies the PostgreSQL database and authentication. Zod validates inputs, and jsPDF and XLSX generate reports. All modules read from the same database, so a change made in one place appears everywhere.

5.1.1 USER INTERFACE AND ACCESS

Users sign in through the login screen and land on a dashboard that matches their role. Coaches, analysts, management staff and players each see the functions relevant to them. The dashboard gives a quick overview of teams, players and matches, and the navigation leads to the analytics, selection, pitch intelligence, opponent analysis, matches, live match, tournament, development, AI assistant and report pages. The login and dashboard screens are shown in Figures 5.1 and 5.2.



Crick10

Dashboard

Teams & players

Playing XI

Pitch analysis

Opponent analysis

Player development

Analytics

Reports

Rahul Sharma

Coach - National High Performance C...

Sign out

Teams & Players

Real T20 records for the last three years, plus career context.

All Teams (200)

IN India #268

England #269

AU Australia #260

NZ New Zealand #247

ZA South Africa #243

West Indies #238

Pakistan #232

SL Sri Lanka #228

All Roles

Batters

Bowlers

All-Rounders

Wicketkeepers

Search player name...

Player	Role	Age	Mat	Runs	SR	Avg	Wkts	Econ	Impact	Profile	
Wenindu Hasaranga	SL Lanka - Right hand bat - Right arm legbreak	All-Rounder	29	48	630	135.9	17.7	68	6.89	+113.3	View >
Abhishek Sharma	India - Left hand bat - Left arm orthodox	Batter	26	21	836	198.4	35.3	3	9.33	+105.9	View >
Kuldeep Yadav	India - Left hand bat - Left arm wrist spin	Bowler	31	25	16	75.8	5.8	39	6.89	+101.9	View >
Andre Russell	West Indies - Right hand bat - Right arm fast	All-Rounder	38	35	580	197.1	23.2	29	8.45	+99.4	View >
Glenn Maxwell	Australia - Right hand bat - Right arm offbreak	All-Rounder	37	41	800	166.7	29.8	22	7.78	+96.8	View >
Matthew Forde	West Indies - Right hand bat - Right arm fast-medium	All-Rounder	24	12	80	138.5	15.8	18	8.88	+96.7	View >
Shaheen Shah Afridi	Pakistan - Left hand bat - Left arm fast	Bowler	26	36	90	128.6	19.8	52	7.38	+95.0	View >
Ben Stokes	England - Left hand bat - Right arm fast-medium	All-Rounder	35	16	320	138.6	26.7	14	8.88	+94.8	View >
Varun Chakravarthy	India - Right hand fast - Right arm legbreak	Bowler	35	16	4	88.8	4.8	25	6.58	+94.4	View >

Crick10

Dashboard

Teams & players

Playing XI

Pitch analysis

Opponent analysis

Player development

Analytics

Reports

Rahul Sharma

Coach - National High Performance C...

Sign out

Playing XI Recommendation

Constraint-aware optimization engine with opponent matchup intelligence for T20 Internationals.

MATCH FORMAT

20 overs per side - Optimized for strike-rate, death-overs economy & powerplay matchups

T20 FORMAT

MY TEAM

IN India (Rank: No. 1 - 268 pts)

OPPONENT TEAM

AU Australia (Rank: No. 3 - 260 pts)

MATCH VENUE (0)

Pace & Seam

Spin Deck

High Score

Balanced

AU

Targeting Opponent: Australia

Rating #260

T20

Squad Profile: 25 Players (3 Left-Hand Batters, 6 Seam Bowlers, 2 Spinners)

Format Metrics: Boundary Strike Rate & Death Variations

Pitch Dynamics

Live Impact

Grass Cover (Seam)

40%

Moss/Low Level (Swing)

45%

Hardness & Carry

65%

Surface Cracks (Spin)

25%

Bounce Index

60%

Dew Factor (2nd Innings)

T20 SQUAD BALANCE

11 / 11 Selected

IN India Starting XI vs Australia

T20 (20-Over) Setup: Spin-friendly conditions. Designated spin destroyers deployed at No. 5; multiple spin options selected.

Opponent & Surface Tuned

Opponent Strategy vs Australia: Solidified top order with pace-resilient strokeplay against express bowling.

1

Abhishek Sharma

Spin Destroyer

Top-Order

Left hand bat - Left arm orthodox

635 runs - 188.4 SR

+106.5

Powerplay Opener: Sets tempo against the new ball (Abhishek Sharma)

2

Yashasvi Jaiswal

Pace & Seam Hunter

Top-Order

Left hand bat

723 runs - 163.9 SR

+88.8

Opening Partner: Complements Powerplay start

Match Status

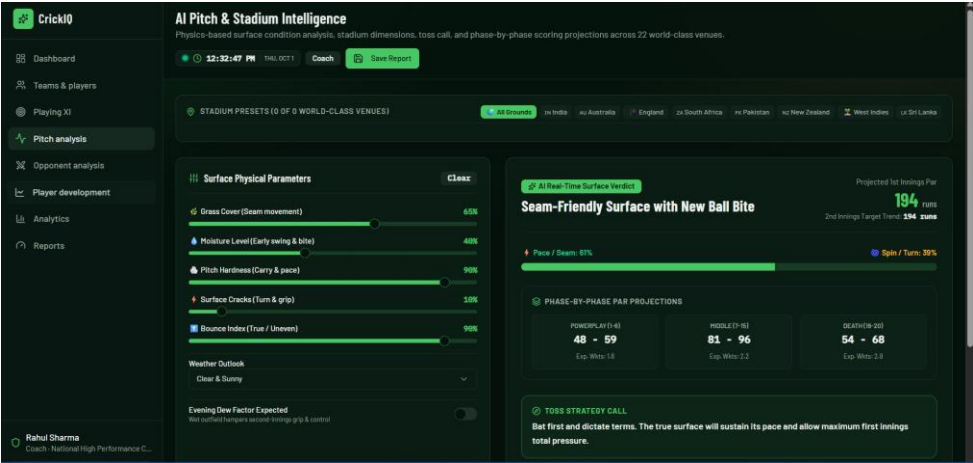
IN India vs Australia

1st Innings

India: 100/0 (10.0 overs)

Australia: 100/0 (10.0 overs)

41



CONCURRENT USERS	AVG. SYNC LATENCY	MESSAGES SENT	MESSAGES RECEIVED	DATA SENT	DATA RECEIVED
2	44.51 ms	48	53	1.21 KB	3.74 KB
5	29.74 ms	120	505	3.02 KB	24.53 KB
10	18.52 ms	240	2,225	6.04 KB	96.96 KB
20	12.68 ms	480	9,179	12.29 KB	465.83 KB

5.2 INSPECTION FEATURE FUNCTIONAL VALIDATION

Table 5.4 Inspection Workflow Functional Validation

ID	TEST	EXPECTED BEHAVIOR	EVIDENCE / RESULT
I1	Create inspection and location	Location appears in the location tree	Supported by implemented CRUD and captured UI
I2	Two collaborators add observations	Both authored cards shown with online/offline state	Observed in supplied screenshots
I3	Complementary observations	Relationship, confidence, explanation and combined text displayed	Observed: 92% complementary example

14	Contradictory observations	Potential contradiction displayed without choosing factual winner	Observed: 95% contradiction example
15	Human resolution	Reviewer decision persisted and status reflected	Observed “KEPT BOTH” state; controller persists decision metadata
16	Partial finalization	Finalization count updates but PDF stays locked	Observed 1/2 Finalized with export locked
17	All users finalized	Inspection becomes finalized and export becomes available	Observed 2/2 Finalized with Export PDF enabled
18	PDF generation	Export groups observations by location and includes resolution	Confirmed by supplied sample export PDF

The captured A Block example demonstrates the complementary path. Two collaborators contribute related descriptions of a crack in Room 101. The

semantic panel classifies the pair as complementary, explains that one statement contributes exact location and the other contributes severity, and proposes a combined sentence. This matches the implemented COMBINE action and shows how AI assistance can improve completeness without discarding either observation.

The B Block example demonstrates the safety-critical contradiction path. One collaborator states that the entrance fan is broken, while another states that the entrance fan is fine and a different fan is damaged. The interface reports a potential contradiction with 95% confidence and explains the incompatibility. The user retains both observations, and that human outcome is later represented in the exported PDF. The system therefore exposes disagreement instead of silently rewriting it.

The finalization screenshots validate the approval gate. At 1/2 finalized, report status shows PDF Export as locked. At 2/2 finalized, the workspace is marked FINALIZED, the status message indicates that all collaborators have finalized and the Export PDF control becomes available. This behavior corresponds directly to the backend report-status and finalizeReport logic.

5.3 SEMANTIC RESEARCH EVALUATION FROM THE APPROVED PAPER

The approved conference paper evaluates the broader semantic-triage framework on a separate 300-pair, two-annotator benchmark. That evaluation is not identical to the deployed Gemini-only Inspection service and is therefore reported separately rather than being presented as a production metric. The paper’s full-cascade evaluation reported Cohen’s kappa of 0.800, 80.7% LLMavoidance coverage, 87.3% contradiction recall and 75.7% four-class classification accuracy.

The contradiction-recall result is particularly relevant to the project’s policy because missed contradictions are more consequential than unnecessary review prompts. The research design therefore favors conservative escalation. The implemented Inspection feature follows the same governance principle even

though its current inference path is simpler: contradictions remain human-review items and the model is prohibited from selecting the “true” inspector statement.

Table 5.5 Approved-Paper Semantic Triage Evaluation

METRIC	REPORTED VALUE	INTERPRETATION
Two-annotator agreement (Cohen’s kappa)	0.800	Substantial agreement in the 300-pair evaluation
Classification accuracy	75.7%	Overall four-class performance of the evaluated cascade
Contradiction recall	87.3%	Sensitivity to the safety-critical contradiction class
LLM-avoidance coverage	80.7%	Share of pairs handled without LLM use in the paper’s cascade

5.4 DISCUSSION

The results support the architectural separation used by the project. Yjs solves the availability and convergence problem, while the semantic layer addresses a different class of content risk after observations have been collected. Because the semantic service cannot modify Yjs merge semantics, a temporary AI failure does not prevent users from synchronizing their work. The controller also continues with remaining observation pairs if one AI call fails.

The Inspection workflow adds governance that a general editor does not normally provide. Semantic decisions are durable records, finalization is userspecific, and export is a collective action. This is important in reports where collaborators may be professionally accountable for their observations. The sample PDF shows that unresolved factual disagreement can remain visible as a retained conflict rather than being obscured by generated prose.

The principal limitation is that the final implementation does not yet deploy every component of the research paper’s lightweight cascade. The production

semantic service currently relies on pair filtering plus Gemini, so its cost, calibration and generalization must be assessed separately from the approved paper’s SBERT/NLI/LLM evaluation. The current report therefore keeps research results and implementation validation distinct.

5.5 LIMITATIONS

- The synchronization measurements come from a development environment and do not establish production-scale capacity.
- The highest reported load-test concurrency was 20 simulated users; longerrunning sessions and larger documents were not exhaustively evaluated.
- Network evaluation focused on disconnection/reconnection and did not systematically characterize sustained packet loss, extreme latency or prolonged low-bandwidth conditions.
- The implemented Inspection semantic service depends on an external Gemini API and may vary across model versions, prompts and service availability.
- Confidence returned by the model is an application aid rather than a statistically calibrated probability unless separately calibrated on domain data.
- Pairwise comparison is location-local and may miss long-range semantic dependencies between observations in different locations or across a long report.
- The supplied functional screenshots demonstrate representative Inspection behavior; a larger user study is still required to evaluate usability and review workload in real field inspections.

CHAPTER 6 CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This project designed and implemented an offline-first collaborative platform in which document availability and replica convergence are handled by Yjs CRDTs, while application-level semantics remain a separate concern. Local IndexedDB persistence enables continued editing during disconnection, WebSocket synchronization restores shared state after reconnection, and MongoDB provides server-side persistence and queryable application data.

The major final-stage addition is the Inspection workspace. It converts the semantic-conflict concept from the approved paper into a concrete collaborative workflow built around locations, authored observations, four-class semantic analysis, human decisions, participant finalization and controlled PDF export. The system does not allow the AI to declare which inspector is factually correct. Instead, it classifies the relationship, explains its reasoning, suggests a minimally invasive action and preserves human authority over contradictions.

The implemented finalization rule further strengthens accountability: semantic review must be complete before users can finalize, and the report cannot be exported until all required participants have finalized. The sample export confirms that the resulting PDF can preserve both observations and the selected resolution. Within the evaluated development environment, the existing project tests also reported reliable offline recovery, 100% recorded merge success and consistent final replicas. The project therefore demonstrates how CRDT synchronization, AI assistance and human review can coexist without conflating state convergence with semantic correctness.

6.2 FUTURE WORK

- Integrate the full lightweight cascade described in the approved paper, using structural filtering, Sentence-BERT similarity and bidirectional NLI before selective LLM escalation.

- Calibrate confidence and routing thresholds using a larger inspection-specific benchmark rather than relying only on general-language examples.
- Add richer evidence handling for factual disputes, including photographs, annotations, measurements, timestamps and links to inspection standards.
- Extend semantic analysis from pairwise location-local observations to longer dependency chains and cross-location consistency checks.
- Provide configurable roles for inspector, reviewer and approving authority, with digital signatures and immutable audit export where required.
- Perform field studies with real inspection teams to measure review workload, user trust, false-positive burden and time-to-final-report.
- Expand scalability and network testing to larger user counts, larger documents, longer offline intervals, packet loss and variable bandwidth.
- Explore on-device or privacy-preserving semantic models for deployments where inspection text cannot be sent to a third-party model service.
- Investigate hybrid or peer-to-peer synchronization to reduce dependence on a central WebSocket relay while preserving persistence and access control.

APPENDIX 1 INSPECTION DATA MODEL AND STATUS RULES

This appendix summarizes implementation details extracted from the final project source code. It is included to document the exact final-stage Inspection behavior without overloading the main chapters.

Table A.1 Inspection Rules Implemented in Source Code

AREA	IMPLEMENTED RULE
Inspection status	DRAFT, SYNCHRONIZING, REVIEW_REQUIRED, READY_TO_FINALIZE, FINALIZED
Observation connectivity	online or offline stored with observation snapshot
Semantic relationship	DUPLICATE, COMPLEMENTARY, CONTRADICTION, INDEPENDENT

Human decision	PENDING, ACCEPTED, REJECTED, KEPT_BOTH, KEPT_ONE, EDITED
Candidate filtering	Observation text must contain at least five trimmed characters
AI retry	Up to five retries for HTTP 503 with exponential backoff
Finalization prerequisite	No PENDING non-independent semantic result
Complete finalization	Owner plus all unique collaborators recorded in finalizedBy
PDF availability	Only when inspection is effectively finalized
Export contents	Inspection metadata, observations and semantic results with nonpending decisions

APPENDIX 2 DETAILED TEST CHECKLIST

Table B.1 End-to-End Project Verification Checklist

ID	AREA	CHECK
B1	Authentication	Register/login and open protected dashboard
B2	Document creation	Create a new collaborative document
B3	Real-time edit	Open same document in two clients and verify remote update
B4	Offline edit	Disconnect one client and continue typing
B5	IndexedDB persistence	Refresh/close while offline and verify local content remains
B6	Reconnection	Restore network and verify CRDT convergence
B7	Room isolation	Edit two independent document rooms and verify no cross-room updates
B8	Inspection creation	Create an inspection and verify owner access
B9	Share inspection	Invite collaborator with editor/viewer permission
B10	Location creation	Create multiple inspection locations
B11	Observation snapshot	Add observations and verify stored author/connectivity metadata

B12	Complementary classification	Create compatible observations and verify combined text suggestion
B13	Contradiction classification	Create incompatible observations and verify human-review prompt
B14	Human decision	Record accepted/kept-both/other resolution and verify persistence
B15	Review gate	Leave a required result pending and verify finalization is blocked
B16	Partial finalization	Finalize one user and verify PDF export remains locked
B17	Complete finalization	Finalize all users and verify FINALIZED status
B18	PDF export	Export and verify locations, observations and resolved conflicts appear
B19	Deletion cleanup	Delete observation and verify related stale semantic results are removed
B20	Research log	Verify analysis/decision/finalization events are logged

REFERENCES

- [1] C. Sun and C. A. Ellis, “Operational transformation in real-time group editors: issues, algorithms, and achievements,” in Proc. ACM Conf. Computer Supported Cooperative Work (CSCW), 1998, pp. 59-68. doi: 10.1145/289444.289469.
- [2] N. Preguica, J. M. Marques, M. Shapiro, and M. Letia, “A commutative replicated data type for cooperative editing,” in Proc. 29th IEEE Int. Conf. Distributed Computing Systems (ICDCS), 2009, pp. 395-403. doi: 10.1109/ICDCS.2009.20.
- [3] M. Shapiro, N. Preguica, C. Baquero, and M. Zawirski, “Conflict-free replicated data types,” in Proc. 13th Int. Conf. Stabilization, Safety, and Security of Distributed Systems (SSS), 2011, pp. 386-400. doi: 10.1007/978-3-642-24550-3_29.
- [4] M. Kleppmann and A. R. Beresford, “A conflict-free replicated JSON datatype,” IEEE Trans. Parallel Distrib. Syst., vol. 28, no. 10, pp. 2733-2746, 2017. doi: 10.1109/TPDS.2017.2697382.
- [5] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in Proc. ACM SIGPLAN Int. Symp. New Ideas, New Paradigms, and Reflections on Programming Software (Onward!), 2019, pp. 154-178. doi: 10.1145/3359591.3359737.

- [6] N. Schiefer, G. Litt, and D. Jackson, “Merge what you can, fork what you can’t: managing data integrity in local-first software,” in Proc. 9th Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2022, pp. 24-32. doi: 10.1145/3517209.3524041.
- [7] G. Litt, S. Lim, M. Kleppmann, and P. van Hardenberg, “Peritext: A CRDT for collaborative rich text editing,” Proc. ACM Hum.-Comput. Interact., vol. 6, no. CSCW2, Art. 531, 2022. doi: 10.1145/3555644.
- [8] J. Gentle and M. Kleppmann, “Collaborative text editing with Eg-walker: Better, faster, smaller,” in Proc. 20th European Conf. Computer Systems (EuroSys), 2025, pp. 311-328. doi: 10.1145/3689031.3696076.
- [9] M. Weidner, J. Gentle, and M. Kleppmann, “The art of the Fugue: Minimizing interleaving in collaborative text editing,” IEEE Trans. Parallel Distrib. Syst., vol. 36, no. 11, pp. 2425-2437, 2025. doi: 10.1109/TPDS.2025.3611880.
- [10] G. Semenov and V. Aksenov, “Semantic conflict model for collaborative data structures,” in Proc. 13th Int. Workshop on Principles and Practice of Consistency for Distributed Data (PaPoC), 2026, pp. 25-31. doi: 10.1145/3806077.3806700.
- [11] N. Gu, J. Xu, X. Wu, J. Yang, and W. Ye, “Ontology based semantic conflicts resolution in collaborative editing of design documents,” Advanced Engineering Informatics, vol. 19, no. 2, pp. 103-111, 2005. doi: 10.1016/j.aei.2005.05.005.
- [12] D. Yang, A. Halfaker, R. Kraut, and E. Hovy, “Identifying semantic edit intentions from revisions in Wikipedia,” in Proc. 2017 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 2000-2010. doi: 10.18653/v1/D17-1213.
- [13] S. R. Bowman, G. Angeli, C. Potts, and C. D. Manning, “A large annotated corpus for learning natural language inference,” in Proc. 2015 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 632-642. doi: 10.18653/v1/D15-1075.
- [14] A. Williams, N. Nangia, and S. R. Bowman, “A broad-coverage challenge corpus for sentence understanding through inference,” in Proc. NAACL-HLT, 2018, pp. 1112-1122. doi: 10.18653/v1/N18-1101.
- [15] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in Proc. EMNLP-IJCNLP, 2019, pp. 3982-3992. doi: 10.18653/v1/D191410.
- [16] J. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, “FEVER: a large-scale dataset for fact extraction and verification,” in Proc. NAACL-HLT, 2018, pp. 809-819. doi:

10.18653/v1/N18-1074.

- [17] M. Lee, P. Liang, and Q. Yang, “CoAuthor: Designing a human-AI collaborative writing dataset for exploring language model capabilities,” in Proc. CHI Conf. Human Factors in Computing Systems, 2022, Art. 388, pp. 1-19. doi: 10.1145/3491102.3502030.
- [18] Q. Ruan, I. Kuznetsov, and I. Gurevych, “Re3: A holistic framework and dataset for modeling collaborative document revision,” in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024, pp. 4635-4655. doi: 10.18653/v1/2024.acl-long.255.
- [19] L. Gao et al., “RARR: Researching and revising what language models say, using language models,” in Proc. 61st Annual Meeting of the Association for Computational Linguistics (ACL), 2023, pp. 16477-16508. doi: 10.18653/v1/2023.acl-long.910.
- [20] P. Manakul, A. Liusie, and M. J. F. Gales, “SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models,” in Proc. 2023 Conf. Empirical Methods in Natural Language Processing (EMNLP), pp. 9004-9017. doi: 10.18653/v1/2023.emnlp-main.557.